Workforce scheduling with constraint logic programming

N Azarmi and W Abdul-Hameed

Recently, constraint logic programming (CLP) has been successfully applied to many industrial scheduling and resource allocation problems. This paper presents results of applying CLP to a typical industrial workforce scheduling problem. A number of approaches to the problem have been developed and implemented in CLP languages CHIP and ElipSys. A detailed analysis of various experiments is also reported.

1. Introduction

The interest in constraint logic programming (CLP) is due to its advantage over traditional operations research (OR) techniques for solving problems such as scheduling and resource allocation and planning. This class of problems can be viewed as constraint satisfaction problems (CSPs). A CSP is a search problem with constraints, where a constraint expresses a relationship among one or more objects of a particular domain. CSPs and their optimisation extension, i.e. constraint optimisation problems (COPs), are usually hard combinatorial problems for which there are no general and efficient algorithms.

The traditional approach to solving CSPs is to write specialised programs in procedural languages with the possible use of relevant OR techniques. Such approaches entail a long system-development time, and are difficult to modify and maintain. On the other hand, CLP languages attempt to overcome these difficulties by combining declarative aspects of logic programming with powerful constraint handling techniques. In practice CLPs have been shown to provide rich constraint modelling languages, short development time, ease of modification, powerful built-in constraint-solving engines and quick adaptation to new or changing requirements. They also provide the flexibility and efficiency needed to solve industrial and commercial CSPs and COPs. These latter features of CLPs are expected to be significantly improved with the current focus in research and development on the incorporation of OR techniques within the CLP framework [1].

The research which is reported in this paper explores the application of CLP languages to solve the workforce scheduling problem. The work has resulted in the development of a CLP system for workforce shceduling [2]. The flexibility of the CLP allowed for a number of approaches to the problem (in the form of prototype systems) to be developed and tested. The first system was developed in CHIP which is a CLP language based on PROLOG whose constraint solving engine is based upon an arc-consistency technique for CSPs with variables ranging over finite integer domains [3]. CHIP was originally developed at ECRC. This system was further extended within another CLP framework, namely ElipSys which is an extension of CHIP and provides the option of exploring parallel execution to enhance the performance of application programs. ElipSys supports — OR-parallelism, independent AND-parallelism and data parallelsim [4]. ElipSys was also developed at ECRC.

The first prorotype was based on a constraint-based ‘tour generation’ (TG) algorithm and a heuristic schedule repair technique. The extended prototype system was based on a constraint-based ‘compact generation’ algorithm, an extension to TG, which solves multi-time-constrained travelling salesman problems. the extended prototype was further enhanced to include parallel execution [5]. Several parallel prototype experiments were conducted. The full findings of this research are documented in Abdul-Hameed [6]. This paper only reports on the main algorithms employed in the two deeloped prototypes.

2. Workforce scheduling problem

The workforce scheduling problem (WSP) addresses a situation consisting of a number of engineers with vehicles and right stores who have to perform a number of jobs. The WSP is essentialy a multi-time-constrained travelling salesman problem (TCTSP).

A travelling salesman problem (TSP) involves selection of a shortest route among a number of locations to be visited subject to the constraints that the person must return to the starting location at the end of his/her journey, and that each location can only be visited once. The TCTSP is a TSP

which in addition involves a number of temporal constraints which need to be satisfied, e.g. location A must be visited by a certain time of the day. The WSP is a multi-TCTSP as it involves a number of engineers where each engineer's tour represents a TCTSP.

The objective in solving the WSP is to maximise the amount of work done and to minimise the amount of travel.

This section describes BT's work-force scheduling problem as studied for this research. Two different types of WSPs are considered in this work. The first (described in this paper) is extracted from real life scenarios, while the other (described in Brind et al [7] and referred to as a vehicle routeing problem — VRP) is randomly generated by the University of Strathclyde. A more detailed description of the considered WSP can be found in Baker and Purohit [8].

2.1 Problem statement

There are a given number of jobs to be performed during a specified time period, and a given number of engineers who can do these jobs. Each job is to be performed at a particular location, and each engineer has a base location. The objective is to:

- allocate as many jobs as possible,
- minimise the travel time of each engineer.

The considered real-life data sets (RDs) are under-resourced as there are insufficient engineers to complete all the jobs [2]. Two data sets are used, RD-250-118, and RD-434-120, where 250 and 434 are the number of jobs in the respective data sets, and 118 and 120 are the numbers of engineers. The ratio of 250 jobs to 118 engineers (in the RD-250-118 data set) is misleading as the time needed to complete a job can be very large. Similar argument applies to RD-434-120 data set.

2.2 Problem constraints

Each engineer has a:

- base location,
- start time,
- end time.

An engineer cannot start working earlier than his start time, implying that he cannot travel to his first job before this time. An engineer must start from his base location, do all the jobs in his schedule, and return to his base no later than his end time.

Certain jobs are constrained to be done at a particular time of the day:

- 'First' — this must be the first job done in an engineer's schedule,
- 'Last' — this must be the last job to be done in an engineer's schedule,
- 'Morning' — this has to be started in the morning, but it can run into, and be completed in the afternoon,
- 'Afternoon' — a job of this type must start after midday,
- 'All-day' — this job can be done at any time.

In addition, the allocation of a job to an engineer is limited to the set of engineers who have the appropriate technical qualifications to do that job.

2.3 Calculating job duration

Each job has a base duration. Each engineer has a skill level which determines how quickly he can do a job. This skill level is constant over all jobs. Obviously a skilful engineer will complete the same job in a shorter time than a less skilled engineer. The time taken to complete a job (ActualDuration) is given as:

$$\text{ActualDuration} = \text{BaseDuration} \times (\text{Skill}/10)$$

2.4 Calculating travel time

The travel time (TT) between two locations whose coordinates are (X_1, Y_1) and (X_2, Y_2) is calculated by the function specified below [8]:

if $|X_1\text{-}X_2| > |Y_1\text{-}Y_2|$
then $TT = (|X_1\text{-}X_2| + (|Y_1\text{-}Y_2|/2))/8$
else $TT = (|Y_1\text{-}Y_2| + (|X_1\text{-}X_2|/2))/8$
end.

2.5 Solution cost

The quality of the solution produced is determined by the function below. The objective is to minimise this function:

$$\text{Cost} = -\sum_{i=1}^{\text{NoE}} TT_i + \sum_{j=1}^{\text{NoU}} (Dur_j + \text{Penalty})$$

where: NoE is the total number of engineers used,
NoU is the total number of unallocated jobs,
TT_i is the total travel time of engineer i to travel from his base to all the jobs in his schedule, and return to base,
Dur_j is the base duration of job j,
Penalty is a constant amount currently set to 60.

The WSP as specified in Abdul-Hameed [2] and Baker and Purohit [8] also allows for the 'overtime' which must be minimised. However, overtime has been disallowed for the experiments reported in this paper.

3. Prototype 1 — tour generation and repair

The WSP (workforce scheduling problem) described in section 2 was solved through the implementation of two separate phases. The first phase concentrated on producing a good solution tour (with respect to the cost function) in which jobs are allocated to engineers or identified as being unallocated. A solution tour consists of one workday schedule for each engineer (an engineer may have an empty schedule). The second phase concentrated on scheduling unallocated jobs produced by the first phase. This is done through an iterative repair method which attempts to shuffle the current engineer schedules, with minimal disruption, to make room for the unallocated jobs. The first phase is called 'tour generation', and the second phase is termed 'repair'. These algorithms were implemented in the CLP language CHIP and are described briefly in this section. For further details regarding implementation the reader is referred to Abdul-Hameed [2].

It should be noted that the WSP is not a classical CSP but a constraint optimisation problem (COP), i.e. CSP with an associated optimisation function — the optimisation function being the cost function specified in section 2.5. CHIP's constraint satisfaction engine was significant in the generation of valid solutions. For the formulation of the WSP in CHIP, see Appendix A.

3.1 Tour generation algorithm

The tour generation (TG) algorithm took a job-centred approach in that it assigned an engineer to a job. It deals with one job at a time, allocating to each job an engineer and a start time. The assignment is made so that an individual engineer's tour is built from the beginning of his workday onwards. The algorithm considers jobs for allocation in the order of: 'first', 'morning', 'afternoon' and 'last' respectively. (Note that 'all-day' jobs have been equally distributed between 'morning' and 'afternoon' jobs.) This means that all morning jobs must be considered for allocation before any attempt is made to allocate afternoon jobs, and so on. This also has the advantage of avoiding expensive backtracking. For example, if an afternoon job was assigned first followed by a morning job which runs into the afternoon, then a temporal constraint is violated. Consequently, back-tracking will be initiated to the point of assignment of the afternoon job, and new start times will be assigned to that afternoon job. Given the large range of start times, this would be computationally expensive. Moreover, there are the thrashing symptoms [3] to overcome before selecting a satisfactory start time for the afternoon job.

The basic steps of the algorithm are as follows:

- the choice of engineers and start time values for a job is limited by declaring their respective domain variables (i.e. a CLP modelling technique, see Appendix A),
- constraints are stated to prevent a job being assigned to more than one engineer, and to prevent an engineer having more than one first job and one last job,
- jobs are assigned to engineers by a process called labelling [3].

The TG's labelling algorithm is as follows:

Let Remaining_Unallocated = the set of jobs that have still not been allocated to engineers.

WHILE (Remaining_Unallocated is not empty) DO

1. Select a job (call it X) from Remaining_Unallocated.
2. Attain a list of the engineers qualified for X (call this list Qualified).
3. IF (Qualified is not empty) THEN
   a) select an engineer Y from Qualified,
   b) remove Y from Qualified.
   ELSE
   a) assign X to the dummy engineer,
   b) go to step 5.
4. IF (the engineer Y satisfies previously generated temporal constraints) THEN
   check the validity of engineer's lcoal constraints
   IF (engineer's local constraints are satisfied) THEN
   a. the engineer Y is allocated to the job,
   b. the job is assigned the minimum start time it can have according to the engineer's partially built tour up to this point,
   c. propagate temporal constraints on other as yet unallocated jobs.
   ELSE
   go back to step 3.
   END
   ELSE
   go back to step 3

End

5. Remove the selected job (i.e. job X) from Remaining_Unallocated.

END (WHILE LOOP).

It should be noted that the terminating condition is catered for by the 'else' branch of step 3. The dummy engineer can accept any job and as such is a constraint relaxation step. This prevents the use of backtracking when jobs are not allocatable due to constraint violations.

3.2 Repair algorithm

The tour generation algorithm described above produces a low-cost solution when compared to other techniques (see section 5). To further improve this initial solution, an iterative repair algorithm (similar to the one described in Minton et al [9]) was implemented.

The algorithm focuses on reducing the number of unallocated jobs with the aim of reducing the overall cost. It does this by removing a job from an engineer's current schedule and trying (if possible) to assign it to another engineer. This may allow the first engineer to be assigned some as yet unallocated jobs and reduce their number. However, it cannot always be guaranteed that assigning an unallocated job to an engineer will reduce the overall cost since the travel cost may be increased. However, as was shown in an analysis of the results [2], the repair often caused minimum disruption to the engineers' tours and travel times in the initial solution. The repair algorithm known as 'swap-insert' is briefly described below:

1. Select a job (Unalloc) from the initial list of unallocated jobs (called Unalloc_init).
2 Swap Unalloc with an allocated job (called the Displaced Job) of a qualified engineer, ensuring that all engineer's local constraints are satisfied after the swap operation.
3. If a valid swap operation has occurred, then try to insert the displayed job into a qualified engineer's schedule (possibly re-insert in the same schedule that it was displaced from), so that all constraints are still satisfied.
4. If a valid swap cannot be followed by a successful insertion of a displaced job, then try step 2 with an alternative swap choice (if possible).
5. If an alternative swap in step 4 is not possible, then the first encountered valid swap is accepted and the resulting displaced job is added to the Unalloc_new list.
6. If step 2 cannot be applied, then Unalloc remains unallocated and is removed from Unalloc_init, and inserted in Unalloc_new.
7. Get the next job from Unalloc_init, and repeat the above steps until Unalloc_init is empty.
8. When Unalloc_init is empty repeat the above steps with the current unallocated jobs, i.e. in Unalloc_new list.

One cycle of the algorithm consists of trying to allocate all the jobs in 'Unalloc_init', creating a list of newly unallocated jobs, 'Unalloc_new'. (This list will contain some old unallocated jobs from 'Unalloc_init' where it is not possible to apply step 2 for those jobs). The next cycle repeats the same operations on 'Unalloc_new'. Naturally, another new list of unallocated jobs may be created in the second cycle, so that a third cycle can be performed and so on. Since it is impossible to allocate all the jobs in the BT data sets, the terminating condition of the repair is the number of cycles attempted.

The following observations have been made:

- step 6 is necessary since the repair of the solution in one cycle may allow an unallocated job to be allocated in the next cycle,
- the attempt to reduce the number of unallocated jobs is made through step 3,
- step 5 disrupts the schedule in the hope that this will generate a better opportunity to reduce the number of unallocated jobs — this step is especially needed in the first cycle since it is known that the tour generation phase could not allocate the jobs in 'Unalloc_init' given the previous assignments. Therefore these previous assignments must be altered.

These are only brief descriptions of the algorithms. Further information is contained in Abdul-Hameed [2], including reasons why a repair algorithm was preferred over a CLP branch-and-bound optimisation.

3.3 Results and findings

Styding the labelling algorithm of tour generation (section 3.1) revelas that there are two points where local heuristics can be applied. These are:

- the selection of a job for allocation (step 1 of labelling),
- the selection of an engineer to assign to a job 9step 3 of labelling).

In selecting which job to allocate next, a first-fail heuristic was used. The principle of first fail is to selet the domain variable which is most likely to fail [3]. The justification is that attempting the most difficult parts of the search space first aids in making failures appear and therefore causes pruning to occur early in the search. In our formulation of the WSP, this meant selecting the job which has the fewest number of qualified engineering. Intuitively, this heuristic should help in decreasing the number of unallocated jobs as jobs which can be done by a larger range of engineers have more change of being allocated in the later stages.

A search algorithm described in French [10] as 'branch-and-bound without backtracking' was used to select engineers for assignment to a job. The algorithm expands the search tree by selecting the child node with the least cost. The tour generation modelled this by seleting the engineer that adds the least increment to the global solution cost so far. The component of the cost added being the travel time of the engineer.

Unfortunately, the best local choice is not always the best global choice, which is a drawback of both heuristics. However, such drawbacks are to be expected as these methods are only designed to achieve a good approximation of bloba optimisation by making the best local choices.

Table 1 illustrates the differences that the heuristics make to the quality of the solution. This is highlighted by the 'total cost' column, which shows that applying first fail and selecting the engineer closest reduces the total cost by some 3500 points (see rows (d) and (a)).

From these experiments, it cannot be concluded that the first fail principle is significant in reducing the number of unallocated jobs as the decrease in unallocated jobs in (a) to that of (c) is only slight (however, see section 4.5, Table 3). This suggests that the first fail must be affecting the selection so that jobs with large duration are allocated first. The evidence for this is from the comparison of the 'unallocated cost' column for (a) and (c) which shows that selecting the engineer cloest to a job gives the expected result of reducing the travel cost.

Examining the results of applying the repair algorithm shows it to be efficient in reducing the number of unallocated jobs — by some 35% in relation to the initial solution. Correspondingly there is a notable reduction in the total solution cost (approximately 6%). The increase in travel cost caused by the repair is minimal as shown by the 'average travel to a job' column. This is expected since the travel cost of an engineer's schedule before and afteran inserted job is unaffected, and adheres to the local travel cost minimisation heuristic. The low travel cost is not ensured for the part consisting of the (repair-based) inserted job and its immediate adjacent jobs. Intuitively, the insertion of a job into a tour should therefore cause minimal disruption. This means that the good work of the tour generation phase is still retained — the supporting evidence being the results in Table 1.

4. Prototype 2 — compact generation with a TCTSP solver

The discussion now turns to the extensions made to the tour generation and repair algorithms developed in the initial (CHIP-based) prototype. This section will first describe the extensions made, and then provide a comparative analysis between the initial and extended (ElipSys-based) prototype.

4.1 Compact generation

The compact generation algorithm extends the tour generation algorithm by attempting to compact individual engineer tours. It does this by the application of a time-constrained travelling salesman problem (TCTSP) optimisation algorithm. In a TCTSP, each task could be represented as a node in a graph, where the distance associated with an edge between two tasks is the sum of the duration of the first task, the travel time between the two tasks' locations, and the set up time for the second task if it is executed just after the first task [11]. The modelled problem employs only limited temporal constraint propagation (i.e. using only 'after' temporal relations). The Strathclyde and BT problems do not contain set up times.

The compact generation (CG) algorithm is very similar to the tour generation (TG) algorithm. The main difference is that in CG, when a job is assigned to an engineer, that engineer's previous tour is re-organised so as to have the optimal tour with this job added. It is the role of TCTSP optimisation to ensure the optimality of that engineer's tour. Contrastingly, in TG, when a job is assigned to an engineer,

Table 1 Results of applying trou generation and repair to the RD-250-118 data set.

	Total cost	Travel cost	Average travel to a job	Number unallocated	Unallocated cost	CPU time (seconds)
a) Tour generation (no heuristics)	26849	6949	39.70	75	19900	150
b) Tour generation (closest engineer only)	24732	4666	27.12	78	20066	144
c) Tour generation (first fail only)	24611	6318	35.89	74	18293	150
d) Tour generation (with both first fail and closest engineer heuristics)	23398	4686	26.62	74	18712	120
e) Repair — applied to (d) *	21705	5515	27.43	49	16190	3600

* The same repair ported on to ElipSys takes only ten minutes to achieve the best result. The difference is due to ElipSys using compiled code, while CHIP interprets.

it is simply inserted at the end of that engineer's previous partial tour, which has the natural limitation of possibly being a non-optimal tour.

The basic CG algorithm can be obtained by replacing the step 4 of TG algorithm (section 3.1) with:

```
4.1  Attain Y's tour (call it Y_tour).
4.2  Apply TCTSP optimisation to the set of jobs 'X U Y_tour, to
     give 'Y_tour_opt'.
4.3  IF (Y_tour_opt satisfies all the problem constraints) THEN
       X is assigned to engineer Y, go to step 5.
     ELSE
       go to step 3
     END
```

As in TG, there is scope for the use of first fail at step 1 to select a job which has the fewest number of qualfied engineers as a means of decreasing the number of unallocated jobs. In step 3 the closest engineer to a job can be chosen as a means of lowering the global travel cost. However, the engineer with the closest average location to the job is used for this selection, i.e. the *X* co-ordiantes of all the jobs and the base in an engineer's (partial) schedule are summed and averaged. Similarly, the same is done for the *Y* co-ordinates. These are the average *X*, *Y* co-ordinates of an engineer.

Once more use is made of the concept of a dummy engineer to prevent the undoing of any previous job-to-engineer assignments. The job-to-engineer assignments are held ina dynamically updated structure (Fig 1). The use of constraints is limited to the TCTSP module, which applies them to ensure that valid tours are produced and as an aid to pruning the search space whilst looking for the optimal tour. The desription of ElipSys implementation of the TCTSP optimation module, based on a branch-and-bound algorithm is covered in Appendix B.

4.2 Insertion heuristic for TCTSP

For tours containing a small number of jobs, the branch-and-bound (BB) approach for TCTSP finds an optimal solution in a relatively short time. However, it is quite feasible that some tours contain more than ten jobs. Attempting to optimise such tours with BB takes a long time creating bottlenecks.

To bypass such bottlenecks, a simple insertion algorithm is utilised after a tour reaches a certain size. For example, insertion may be used if a tour contains more than eight jobs. When a ninth job (say *X*) is to be added to the current tours, attempts are made to insert *X* at all possible (valid) points without changing the latest tour order between the first eight jobs. The cheapest valid insertion point is then chosen.

The use of an insertion heuristic should make little difference to the global cost as it is only applied to lengthy tours, which in practice are not many. In addition, prior to the very first insertion, a tour is guaranteed to be optimal as it is produced by the TCTSP module. Therefore, the non-optimal part of a tour is that containing the newly inserted job, and the jobs immediately adjacent to it. Finally, there are other promising approaches which could be used for carrying out a TCTSP optimisation task for tours with a large number of jobs [12].

4.3 Extending the repair

An extended version of the swap-insert repair algorithm, described in section 3.3, is used for the second prototype. The limitation of forcing an unallocated job to be placed only in the position vacated by a displayed job is removed (step 2 of the swap-insert). Instead the unallocated job is inserted into any possible valid position of an engineer's tour. Displacing a job is still required to create room for inserting the unallocated jobs.

Figure 2 shows an allocated job *X* being displaced to make room for the unallocated job *ul*. The job *ul* can then be inserted into the trou at any position, as depicted in Fig 3, provided that this position produces a valid tour.

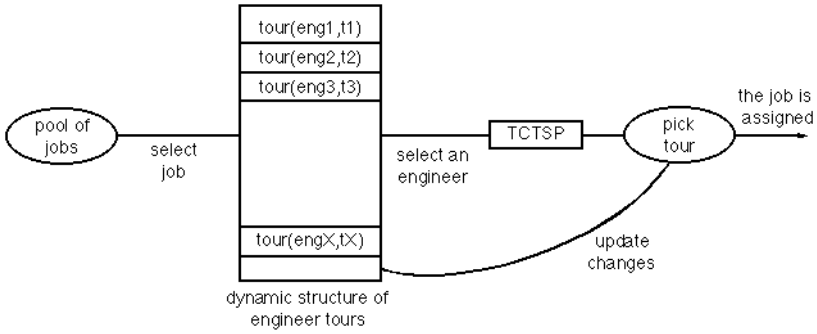


Fig 1 The compact generation architecutre.

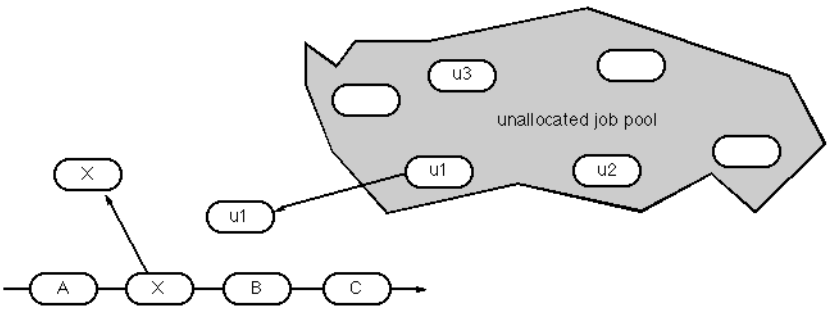


Fig 2 Heuristic repair — removal.

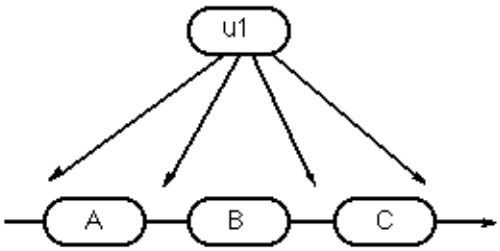


Fig 3 Heuristic repair — insertion.

This extra flexibility does not seem to give a significant improvement as indicated by the results of the experiments in Abdul-Hameed [6].

4.4 Analysis of heuristics used in CG

All the experimental results using CG reported here, unless stated otherwise, apply an insertion heuristic (see section 4.1) in the TCTSP module if the number of jobs to be TCTSP-optimised is greater than seven.

The first set of experiments (Table 2) gives supporting evidence for the use of the insertion heuristic utilised by the TCTSP module in the case where the number of jobs in a tour to be optimised is large enough to produce a bottleneck when generating a solution through CG. The insertion heuristic does not initiate a 'complete' search (i.e. in contrast to the search by the BB), but is, instead, a simple method to approximate an optimal arrangement of inserting a job into a large tour that is already in its optimal configuration.

From Table 2 it is clear that the use of the insertion heuristic lowers the run time drastically (rows (b) and (c) with only a fractional increase in total solution cost (row c)). The reaons for the very small difference in total cost (between rows (a) and (c)) is probably due to the fact that there are very few tours containing more than seven jobs, and they are in an optimal arrangement before the insertion of a job by this heuristic. This should cause minimal disruption to the tour optimality as argued in previous sections.

Attention is now turned to the use of the first-fail principle for selecting the next job to allocate, and the use of an engineer's average location as a good indicator to lowering global travel cost.

Table 3 demonstrates that the use of the first-fail principal for selecting the next job to allocate, and the use of an engineer's average location as a good indicator to lowering global travel cost.

Table 3 demonstrates that the use of the first-fail principal decreases the number of unallocated jobs. Assigning the closest possible engineer to a job, based on that engineer's average location, lowers the global travel cost by a significant amount. Both these heuristics, used on their own, improve on the solution produced without the use of heuristics. The combination of these heuristics gives the best result overall.

4.5 Comparison of tour generation and compact generation

The effectiveness of the CG algorithm is now compared against its predecessor, tour generation.

Table 2 showing application of insertion heuristic in TCTSP module through the CG algorithm applied to RD-250-118

Technique	Total cost	Travel cost	Number unallocated	Cost of unallocated jobs	CPU time (seconds)
a) TCTSP with no insertion	21325	4928	54	16397	1228
b) No TCTSP — insertion heuristic only	22763'	5028	74	17735	70
c) TCTSP with insertion if tour contains more than seven jobs	21409	4892	55	16517	121

Table 3 Showing heuristics used in compact generation (CG) applied to RD-250-118.

Heuristic	Total cost	Travel cost	Number unallocated	unallocated cost	CPU time (seconds)
No first fail, no cloesest engineer	25137	5491	72	19646	70.56
First fail only	22340	5867	55	16473	112.16
Closest engineer only	23807	4459	71	19348	77
First fail and closest engineer	21409	4892	55	16517	121

Table 4 Application of TG and CG to RD-250-118 problem. In CG the TCTSP module with insertion heuristic was used. The figures after the slash in the travel cost column are the average travel time per allocated job.

Technique	Total cost	Travel cost	Number unallocated	Unallocated cost	CPU time (seconds)
TG	23398	4868/26.62	74	18712	121
CG	21409	4892/25.08	55	16517	121

Table 5 Application of TG and CG to the RD-434-120 problem. The TCTSP module was utilised with insertion heuristic. Note that this problem does not have any specialisation/technological constraints (i.e. all engineers could do all jobs).*

Technique	Total cost	Travel cost	Number unallocated	Unallocated cost	CPU time (seconds)
TG	47753	Not available	162	Not available	1546
CG	46716	22941	158	23775	2129

* To reduce the long scheduling execution time, a number of zoning heuristics and parallel execution algorithms (see section 6) were attempted with satisfactory results. For example, these approaches resulted in 50% to 85% decrease in execution time [5,6].

As expected (Table 4), CG produces the better solution by a fairly significant margin. The total travel cost is misleading as CG allocates more jobs and therefore has to travel more. In fact the average travel cost per job indicates that CG reduces the travel time. A by-product of this reduction in travel time is that because engineers spend less time travelling, they have more time to do jobs. Hence the consequent reduction in unallocated jobs.

These results are further verified in their application to the RD-434-120 data set (Table 5).

For a more comprehensive comparison between the two algorithms, the Strathclyde problems were used (Fig 4). The Strathclyde problem-modelling and cost functions were adopted for these experiments, see Brind et al [7, 13]. Briefly, 30-*-* represents under-resourced problems, 50-*-* represents the over-resourced problems, and 40-*-* represents the critically resourced problems, where problems become more constrained as they move from *-0-0 towards *-2-3. Soluble problems are those in which all jobs are allocated, so the total cost only represents the total travel cost. In insoluble problems, some jobs remain unallocated, and therefore the total cost represents both the total travel cost and the total penalty cost for the unallocated jobs. Both TG and CG (in Fig 4) replace the first-fail principle for job selection with the heuristic of selecting the job with the largest duration first. This was because the Strathclyde cost function placed more emphasis on the amount of work not done — consequently, better solutions are produced by reducing the combined duration of unallocated jobs rather than the number of unallocated jobs.

The cost axes show that in the majority of cases, CG produces the better quality (i.e. lower-cost) solution. This is expected since CG has a better facility for lowering travel costs. TG produces the better solutions in the under-resourced problems that have few resource constraints (30-0-0 and 30-0-3). However, as the under-resourced problems become harder (i.e. 30-2-0 and 30-2-3), CG produces the better solutions. This pattern can be explained by CG's ability to compact tours and thereby reduce the number of unallocated jobs. The 50-2-0a and 50-2-0b problems produce strange results showing TG to produce the lower cost. This is unusual since these are over-resourced problems meaning that all jobs can be allocated. In effect, the total costs are purely travel costs. These results should be thought of as anomalies as can be verified by comparison with results of the other '50-*-*' problem sets.

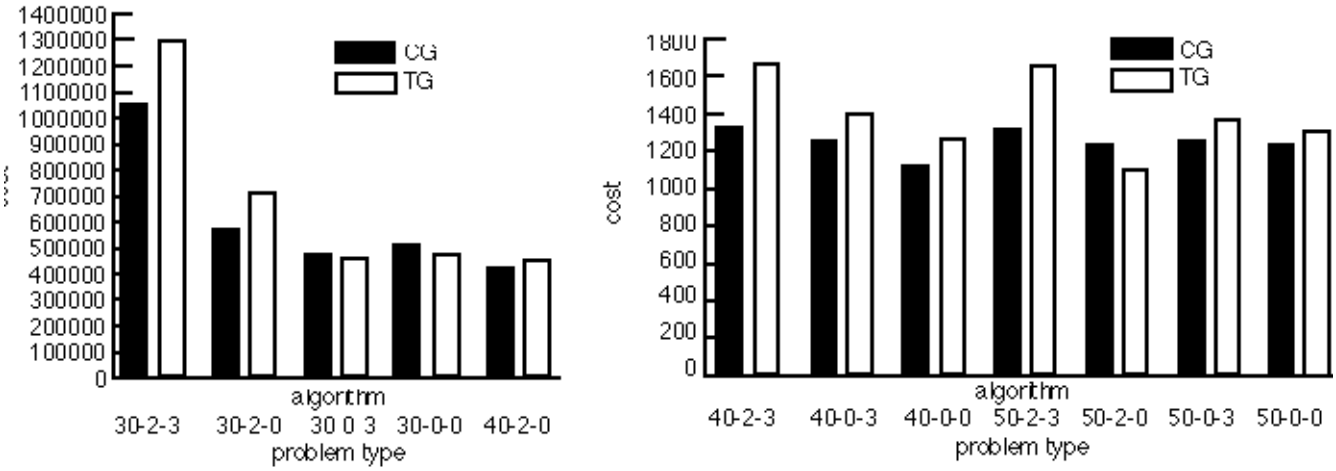


Fig 4 Comparison of CG + repair and TG + repair when applied to Strathclyde problems, where only three cycles of repair were utilised for both algorithms.

The conclusions of these sets of experiments give strong evidence that CG is an improvement over TG. It has a better ability to reduce global travel times which causes engineers to have more time to perform jobs. This emanates in a reduction of unallocated jobs. More importantly, these improvements are shown, in general, under all conditions — under-resourced critically resourced, over-resourced, tightly constrained, without tight constraints, and their combinations.

5. Comparative study with Stochastic techniques

Table 6 shows a comparison of the CLP-based approaches with other techniques that have been applied to the BT WSP — namely the stochastic techniques, genetic algorithms (GA) [14], and simulated annealing (SA) [15]. For the sake of comparison, all of the techniques relax first jobs to be morning jobs and last jobs to be afternoon jobs.

All four techniques produce comparable results. While CLP-CHIP produces the least number of unallocated jobs, the lowest total cost is produced by SA, which is closely followed by CLP-ElipSys. It could be argued that SA is doing less work to avoid increase in the total travel time, whereas CLPs are designed to maximise the number of jobs scheduled. GA and CLPs are similar in that they both contain repair phases. However, the GA repair is an active part of its (initial) solution generation, while CLPs apply the repair in a separate phase. Both GA and CLP repairs aim to reduce the number of unallocated jobs.

A comparison of the optimisation phase reveals mixed results for GA and CLPs. While the GA repair is more effective in lowering the total solution cost, CLP repairs, as shown above, are more successful in decreasing the number of unallocated jobs. However, section 3.3, shows that, in general, the CLP repair is capable of producing an improvement in total cost similar to the GA repair.

It should be understood that the WSP is essentially a constraint optimisation problem (COP) and has relatively few hard constraints. It may therefore be more suited to stochastic techniques such as SA and GA, whereas a more constrained scheduling problem may favour the CLP approach. However, the results of reported experiments with WSP [6] seem to suggest that understanding the underlying structure of the problem could lead to the development of CLP-based solutions to COPs that can compete in performance and quality of solutions with stochastic techniques.

6. Conclusions and further work

This paper has reported on the application of CLPs in solving real-life WSPs. Two prototype systems and their main underlying algorithms, namely tour generation (TG), repair, and compact generation (CG), have been presented. The results analysis of the CG to those of the TG, shows a significant improvement in producing quality schedules. This is due to the use of the TCTSP module i CG which reduces the travel time of individual engineer tours to their optimal states. This results in a global reduction of travel cost which in turn leaves the engineers with more time to perform jobs, thus compacting their schedules. In the case of lengthy schedules, optimality is sacrificed for speed by the use of the insertion heuristic to augment TCTSP. This has shown to be a viable technique in that it reduced execution times considerably with only an insignificant increase in total solution cost.

CLP languages provided a natural, easy-to-use and maintainable program development environment for the formulation of WSPs. The in-built constraint-solving techniques of CLPs significantly facilitated the development of the solution generation (i.e. TG and CG) and optimisation (i.e. repair and BB-TCTSP) algorithms.

The usefulness of the various prototypes implemented showed varying degrees of promise for industrial use. Compact generation, and iterative repair would seem to be

Table 6 Comparison of different techniques (i.e. CLPs and stochastic techniques) aplied to RD-250-118.

Technique		Total cost	Travel cost	Average travel cost per job	Cost of unallocated jobs	Number of unallocated jobs
GA	Before repair	23790	not available	not available	not available	67
	With repair	22570	not available	not available	not available	54
CLP — ElipSys (CG + Repair)	Before repair	21409	4892	25.08	16517	55
	After repair	21292	4902	24.88	16390	53
CLP — CHIP (TG + Repair)	Before repair	22668	5052	27.45	17616	66
	After repair	22241	5269	26.08	16972	48
SA	No repair phase	21050	4390	22.63	16660	56

N.B. This comparison has been done with respect to only one data set (RD-250-118), and the results of SA and GA are averaged over a number of runs.

the most useful and warrant further research. TCTSP could also be adapted to handle specialised cases such as real time scheduling (or rescheduling), i.e. where a schedule has already been assigned and some jobs started, but an alteration to schedules is needed due to some unforeseen event, e.g. an engineer injuring himself and requiring someone to cover for him. This would make TCTSP an interesting option owing to its ability to shuffle tours into an optimal position, given constraints that some jobs cannot be moved once they have already started.

Further investigations, carried out as part of the research programme and reported fully in Abdul-Hameed [3], are as follows.

- CG was extended with parallelism to make gains in both execution times and attaining better-quality solutions1. The parallel compact generation (PCG) gave a demonstration that parallelism could be used to handle the latter point. Although it did not always produce the lowest costing solution, it consistently produced lower travel times when compared with CG. This was because of its selection criteria for assignments of engineers to jobs, which favoured travel-time reduction. Therefore, it is reasonable to assume that alterations could be made to this function which take other factors into account and thereby produce a better solution overall — this is the approach in Caseau and Loppstein [11].

- The application of zoning heuristics demonstrated that it is possible to partition large size data sets (such as RD-434-120) based on specialisation constraints and/or geographical distribution of jobs and engineers. This allowed zoning to be used to produce large reductions in execution times with only miimal disruption to the quality of solutions when compared agtainst sequential non-zoning algorithms (i.e. compact generation). Such reductions in execution times are very important for scalability issues since many search problems (i.e. scheduling problems) display nonlinear increases in execution times with the increase in size of the problem.

Acknowledgements

The authors wish to thank members of the BT-ISU and IC-Parc research teams, in particular R Smith, B Richards and D Pothos for continuous support and fruitful discussions.

Appendix A

Modelling WSP constraints in CLP

The solution representation, for implementation purposes, was modelled in order to reflect the job-centred approach adopted by the tour-generation algorithm. The choice of representation is important as it determines the ease with which constraints can be specified. The following term was chosen to represent the assignment facts related to a job:

```
job_tuple(Jno, St, Adur, Tg, Eng)
```

Table A1 Parameters of 'job_tuple'.

Parameter	Meaning of parameter
Jno	The job identifier.
St	The time at which the job is scheduled to start
Adur	The actual duration of the job based on the engineer assigned to it.
Tt	The travel time frm the engineer's previous location to the present job.
Eng	The identifier of the engineer assigned to the job Jno.

Table A1 explains the parameters of the term 'job_tuple'. Thus the various relations between jobs are expressed through constraints applied to these parameters.

Recapping the definition of the WSP, an engineer's tour is valid provided:

- the engineer has the technical qualifications to perform the job,
- the jobs in his schedule are started at the correct time according to their type,
- all the jobs are done within his workday including travel time from and to his base,
- there is no overlap amongst jobs in his schedule.

The CHIP constraints to model the above WSP constraints are presented below.

Job start times

A job is limited to start at the time specified by its type by declaring the associated 'St' parameter of the 'job_tuple' as a domain variable with the appropriate domain. (For the Strathclyde VRP problems this would be replaced by the time windows.) For example, a morning job must start before midday:

```
St :: 480...719
```

Time is measured in minutes past midnight so that 720 minutes corresponds to 12 noon. The above example shows the domain of 'St' to range from 480 minutes to 719 minutes past midnight where 480 is the earliest start time of any engineer contained in the problem data set.

Selecting qualified engineers

The engineers with the appropriate technical qualifications for a job is contained in the 'job_skills' relations/predicates in the data base. The second argument of the 'job_skills' prediate is a list of qualified engineers. The associated 'Eng' parameter of a job in the 'job_tuple' term is declared as a domain variable, and its domain is this list of qualified engineers. Therefore, the appropriate code segment is:

```
job_skills(Jno,QualifiedEngineers),

Eng :: QualifiedEngineers.
```

There is a further advantage to making this declaration as the set of engineers qualified for a job changes dynamically when certain constraints are imposed, e.g. every time an engineer is assigned a first job, that engineer is removed from the set of engineers belonging to any other first job. This means that the constraint-solving mechanism will alter the domain of 'Eng' appropriately during run time and avoid choices that would lead to failures.

Fitting jobs into a workday

The constraints expressed below prevent a job from being performed outside an engineer's working hours.

$$St \#\geq Eng\_st + Travel\_to\_job \quad (1)$$
$$St + Adur + Travel\_to\_base \# \leq Eng\_et \quad (2)$$

'Eng_st' and 'Eng_et' are the respective start and end time limits of an engineer's work day as contained in the database. Line (1) states that an engineer must allow enough travel time before he can start his first job. Line (2) states that an engineer should complete his last job with enough time to return to his base.

The next problem is to prevent jobs overlapping each other. In other words preventing a job starting before the previous job has completed. Suppose that there are two jobs *A* and *B* such that *B* is forced to start after *A*. The constraints needed are:

pre: Eng_A and Eng_B are assigned values.
no_overlap(St_A, $Adur_A$, Eng_A, St_B, Tt_B, Eng_B):-
$Eng_A == Eng_B$. (3)
$St_A + Adur_A + Tt_B \# < St_B$.

The check at (3) ensures that the constraint is applied only to jobs assigned the same engineer. The constraint at (4) states that job *B* can only start after job *A* has completed and time has been allowed to travel from job *A* to job *B* (Tt_B is the travel time from job *A* to job *B*).

Appendix B

TCTSP optimisation

This module is responsible for attaining the optimal tour for a set of jobs assigned to an engineer. The ElipSys constraints utilised in solving the TCTSP are briefly described next.

Constraint modelling

An engineer's tour is represented by a list of domain variables (Fig B1). The instantiation of these domain variables gives the engineer's ordered tour so that the first value in the list represents the first job the engineer must complete in his tour, and so on. Consequently the length of this list is equal to the number of elements in the set of jobs assigned to him. The domain variables are assigned domains which are subsets of the set of jobs assigned to an engineer.

Fig B1 The tour representing a list of jobs [A,B,C].

For example, suppose that engineer *Y* is to be assigned the set of jobs [J_1, J_2, J_3, J_4]. Then a list of four domain

variables is produced, with each variable's domain being a subset of the set [J_1, J_2, J_3, J_4]. If the jobs J_1...J_4 are all of the same type, then the ElipSys declaration of the domain variables would be:

$[X_1, X_2, X_3, X_4] :: [J_1, J_2, J_3, J_4]$

where X1...X4 are domain variables. In the case where the set of jobs assigned to an engineer have differing temporal constraints, advantage is taken of these restrictions to reduce the complexity of the search space. For example, if the set of jobs assigned to an engineer are [M_1, M_2, A_1, A_2] where M_1, M_2 are morning jobs, and A_1,A_2 are afternoon jobs, then the domain variables X_1...X_4 would be assigned domains as follows.

$[X_1,X_2] :: [M_1,M_2]$,
$[X_3,X_4] :: [A_1,A_2]$.

Thus, in the above example, the possible number of combinations that can be searched is reduced by a factor of four when compared to the first example. Similar constraining of domain variables is applied when there are first and last jobs. Care must be taken with all-day jobs to enusre that they are given the freedom to take any position in the list.

The optimisation of the tour containing morning, all-day, and afternoon jobs may lead to unnecessary exploration of parts of the search space that lead to invalid solutions. For example consider a tour to be made up of jobs in the set {M, Al_1, Al_2, A}, where M, A_l and A represent morning, all-day, and afternoon jobs respectively. A potential tour is then presented by a list of domain variables $[X_1,X_2,X_3,X_4]$ whose domains are defined as:

$X_1 :: [M,Al_1,Al_2]$
$X_4 :: [A, Al_1,Al_2]$
$[X_1,X_3] :: [M,Al_1,Al_2,A]$

Such a declaration of domains makes possible assignments such as $X_1 = Al_1$, $X_2 = Al_2$ and $X_3 = A$. This leads to a dead end since X4 will have an empty domain, and M has not been assigned to the tour. This is due to constraints being placed which prevent a job being done more than one in a tour. A description of this constraint is given later.

This problem is overcome by the use of the built-in ElipSys symbolic constraint sequences/4. For example, to avoid the above assignment, the constraint expressed is:

sequences($[X_1,X_2,X_3]$, M, 1, 1)

which states that there must be one occurrence of M in $[X_1,X_2,X_3]$. However, this does not prevent invalid assignments such as X_2=A and X_3=M being searched. For the data sets studied this omission is not very important. owever, if such as assignment is explored, in the case where X_1=A, and X_n=M where n is large, then much backtracking could be a possiibility. A more satisfactory solution would have been to permit the assignment of afternoon jobs to a tour only if all the morning jobs have been assigned first.

As can be seen the majority of the cases will declare domain variables with overlapping domains. In other words, the same job could be assigned more than once to a tour. To prevent this, a set of inequality constraints is produced. Thus, for a list of domain variables [X_1,X_2,X_3,...,X_n], these could be:

X_1##X_2,..,X_1##X_n, X_2##X_3,..,X_2##X_n,..,
X_{n-1}##X_n

where '##' representd sidequality relation.

The CG algorithm builds up an engineer's tour one job at a time, ensuring that at each stage the optimal tour is produced. With this knowledge the search for a solution can be further constrained. Suppose engineer Y has a current optimal tour Z, where Z=[M_1,M_2,A_3,A_4] and has a travel cost C_1. Let the total duration of the jobs in Z be Z_{dur}, and the sum of the afternoon jobs duration be Z_{dur_af}. Now suppose that an afternoon job X is to be added to Z to produce an optimal tour with cost C_2, and let:

TotalDuration=Z_{dur}+X_{dur}
AfternoonDuration=Xd_{ur_af}+X_{dur}
TotalEngineerTime=Y_{end_time} — $Y_{start\text{-}time}$.

This leads to the constraints below.

TotalDuration ≤TotalEngineerTime	(1)
AfternoonDuration ≤Y_{end_time} - midday.	(2)
C_2 ≤TotalEngineerTime - TotalDuration	(3)
$C_2 \geq C_1$	(4)
C_1 < (TotalEngineerTime - TotalDuration)	(5)

The constraint of line (1) prevents any job being added whose duration is too large for the engineer's available time.

Line (2) ensures that the engineer has enough free time in the afternoon to do any afternoon jobs. In the WSP a similar check cannot be applied to morning jobs since a morning job is allowed to run into the afternoon.

Line (3) bounds the maximum possible value of C_2 (the travel time of the new tour) since the proportion of an engineer's time spent working on the jobs in his tour is known. This constraint is used to prune a branch-and-bound search for the optimal tour (see section 4.2).

At line (4) the information that the travel time of the new tour (C_2) cannot be smaller than the travel time of the old tour (C_1) is used to give a lower bound to C_2. Such a constraint should be useful in the application of branch-and-bound optimisation, which can curtail the proof of optimality if an optimal solution is found.

Finally, the constraint of line (5) tests that there is enough travel time available in an engineer's tour to be able to add the new job X to his current tour Z.

The constraints (1), (2) and (5) prevent a branch-and-bound search from ever occurring, whereas (3) and (4) prune the branch-and-bound search. Together these five constraints decreased the time taken to produce a solution for the RD-250-118 problem from an order of hours to a time of approximately 20 minutes.

Labelling strategy for TCTSP

Labelling in CLPs is the process of assigning values to dimain variables. A list of domain variables is labelled in the order left to right. For example, the domain variables $X_1..X_4$ in the list $[X_1,X_2,X_3,C_4]$ are assigned values in the order X_1,X_2,X_3,X_4. Given the restrictions on variable domains described above, this has the effect of labelling morning jobs before afternoon jobs. This avoids backtracking which may have occurred had the domain variables been assigned in the order afternoon jobs first, then morning jobs.

When a domain variable is assigned a value, the labelling algorithm checks the validity of that assignment. This includes testing temporal constraints, allowing travel time between jobs, and preventing jobs overlapping each other. These are similar to the constraints described in Appendix A.

Branch-and-bound labelling in TCTSP

Branch-and-bound (BB) is an intelligent enumerative search suited for problems consisting of domain variables whose domains range over finite domain natural numbers and a set of constraints between these variables, where the aim is to find the optimal assignment of these variables with respect to some objective function. The application of BB studied in this paper relates to minimising the travel cost of an engineer's tour.

There are a number of ways to apply BB heuristics. One of these is to create an initial solution and restart the search from the beginning. The cost of the initial solution bounds the subsequent search. When a solution is found, or a dead-end encountered, the search back-tracks to the most recent alternative. The search terminates when a solution known to be optimal is found or when all the search space has been explored. This is a depth-first BB variant.

ElipSys implements such as BB heuristic through the built in predicate 'min_max'. Another form of BB is to not restart the search from the beginning, but to backtrack to the last choice point (using the initial cost as a bound to subsequent solutions). ElipSys also provides this facility through the built-in predicate 'minimise'.

The first argument of 'min_max' is a term representing some goal, and the second argument is a cost function associated with this goal. Therefore the utilisation of BB for TCTSP was done by associating, with each domainv ariable of a list representing a tour, a domain variable that represents the cost of travelling to that job. Thus, if there is a list of n domain variables representing an eingineer's tour such as $[X_1,..,X_n]$, the association of a cost domain variable produces the list $[(X_1,C_1),..,(X_n,C_n)]$. Each C_i represents the travel cost to the job identified by the instatiation of C_i. It is this second list that is optimised through min_max applied to the labelling procedure described earlier. The ElipSys call is therefore:

min_max(labelling([$(X_1,C_1),..,(X_n,C_n)$],Cost);

where Cost is a domain variable constrained by the Ci:

Cost # = $C_1 + C_2 +...+ C_n$.

References

1 Lever J et al: 'Constraint logic programming for scheduling and planning', BT Technol J, 13, No 1, pp 73—80 (January 1995).

2 Abdul-Hameed W: 'Workforce scheduling in constraint logic programming CHIP', ISR Technical Report, BT Laboratories (September 1993).

3 Van Hentenryck P: 'constraint satisfaction in logic programming', MIT Press (1989).

4 Prestwich S: 'ElipSys programming tutorial', Technical Report ECRC, 93-2 (1993).

5 Azarmi N and Abudl-Hameed W: 'Workforce scheduling with parallel constraint logic programming, ElipSys', in preparation.

6 Abdul-Hameed W: 'Experimentation with parallel constriant logic programming', MEng Thesis, Dept of Computing, Imperial College, London (also IRESS-Report 94-05, IC-Parc, Imperial College) (June 1994).

7 Brind C, Muller C and Prosser P: 'A study of stochastic search techniques applied to vehicle routeing problems', Technical Report, Strathclyde University, Glasgow (Setpember 1994).

8 Baker S and Purohit B: 'PROJECT MAIN — WORKPACKAGE 3: deliverable-3: formal specification of resource management problems', Internal BT report (1993).

9 Minton S, Johnston M D, Philips A B and Laird P: 'Minimizing conflicts: a heuristic repair method for constraint satisfaction and scheduling problems', AI Journal (1993).

10 French S: 'Sequencing and scheduling — an introduction to mathmatics of the job-shop', Ellis Horwood Publishers, New York (1982).

11 Caseau Y and Koppstein P: 'A co-operative-architecture expert system for solving large time/travel assignment problems;, Ellis Horwood Publishers, new York (1992).

12 Amin S et al: 'A natural solution to travelling salesman problem', BT Eng Journal, 13 (July 1994).

13 Brind C, Muller C and Prosser P: ‘Stochastic techniques for resource management’, BT Technol J, 13, No 1, pp 55—63 (January 1995).

14 Muller C, Magill E H and Smith D G: ‘Distributed genetic algorithms for resource allocation’, Technical Report, Strathclyde University, Glasgow (1993).

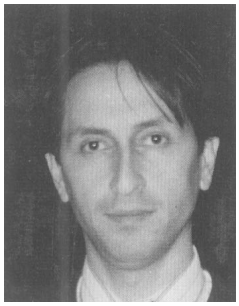
15 Baker S: ‘Applying simulated annealing to the workforce management problem’, ISR Technical Report, BT Laboratories, Martlesham Heath, Ipswich (1993).

16 Munaf D: ‘PROJECT MAIN — WORKPACKAGE 12: realization of parallel logical languages: workforce management in Andorra-I’, ISR Technical Report, BT Laboratories, Martlesham Heath, Ipswich (1993).

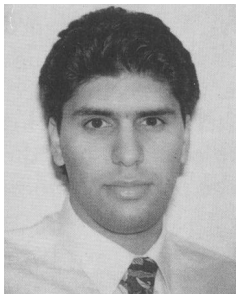
17 McKeown G P, Rayward-Smith V J and Rush S A: ‘Parallel branch-and-bound’, University of East Anglian (1992).

18 Aarts E and Korst J: ‘Stimulated annealing and Boltzman machines’, Wiley, New York (1989).

19 Perry E L: ‘Solution of time constrained scheduling problems with parallel tabu search’, in Proc of DARPA Workshop: ‘Innovative approaches to planning, scheduling and control’, pp 231—239 (1990).



Nader Azarmi received a BSc Honours degree in Computer Science and Mathematics from the University of Manchester in 1982, and an MSc in Computer Studies (AI) in 1985 and an MPhil in Computer Science (Logic Databases) in 1990 from the University of Essex. Since 1992 he has been pursuing a PhD degree in Computer Science (Reactive Scheduling Systems) at Imperial College, London University. In 1988 he took a senior lectureship post in the Computer Science and Electronic Engineering School of Kingston University. Since December 1989 he has been working at BT laboratories on the application of AIP techniques to telecommunications network automatic fault diagnosis, and to resource management.



Watek Abdul-Hameed received a Master of Engineering degree in Software Engineering with first class honours from Imperial College in 1994.

He spent his industrial placement at BT-ISU and has recently joined BT’s Belfast Engineering Centre.